Project Proposal: AirMorse

(Contactless Morse Code Virtual Keyboard via Hand Tracking)

Subject: Python Programming Project
Team Size: Up to 4 members
Target Platform: Desktop Application (Windows / macOS)

---

1. Executive Summary

AirMorse is a lightweight, real-time desktop application developed in Python that provides a completely touchless text input mechanism using an off-the-shelf laptop webcam. By leveraging computer vision and hand tracking models, the system continuously tracks hand landmark coordinates to interpret five-finger pinch gestures into International Morse Code:

- **Index–Thumb pinch:** Emits a **Dot** (`.`).
- **Middle–Thumb pinch:** Emits a **Dash** (`-`).
- **Ring–Thumb pinch:** Inserts a **Space** delimiter.
- **Pinky–Thumb pinch:** Triggers a **Backspace** action.

Collected Morse sequences are decoded on the fly into alphanumeric characters (A–Z, 0–9) and injected directly into active operating system windows (such as text editors and web browsers) while providing visual feedback on an on-screen virtual keyboard HUD. AirMorse serves as an innovative assistive technology for individuals with motor impairments, hands-free sterile workflows (e.g., surgical or culinary environments), and next-generation human-computer interaction.

2. Why This Project? (Team Benefits)

- **High Grading Impact:** Features real-time skeletal landmark rendering, interactive visual feedback on a virtual keyboard HUD, synchronized auditory Morse feedback beeps, and system-level keystroke injection across third-party applications—guaranteeing an impressive and dynamic live presentation demo.
- **Manageable Learning Curve:** Built upon Google's pre-trained MediaPipe Hands framework to extract 21 3D landmarks instantly. The team avoids dataset collection and training deep neural networks from scratch, focusing instead on spatial geometry, $\arg\min$ optimization, and Finite State Machine (FSM) threshold logic.
- **Low Computational Overhead:** Operates smoothly at 30–60 FPS on standard laptop CPUs without requiring dedicated GPU hardware.
- **Modular Architecture:** The codebase cleanly bifurcates into four isolated modules (Vision Pipeline, Gesture Logic & FSM Core, UI/Virtual Keyboard HUD, and System Keystroke & Audio Integration), ensuring straightforward task distribution and Git collaboration.

3. Core Features & Capabilities

1. **Normalized Five-Finger Pinch Gesture Engine:** The vision sub-system continuously tracks 3D Cartesian coordinates (x, y, z) of 21 keypoints from Google MediaPipe Hands,

focusing on the thumb tip (P_4) and the four remaining fingertips: Index (P_8), Middle (P_{12}), Ring (P_{16}), and Pinky (P_{20}).

- *Scale Invariance:* To ensure invariant classification irrespective of the user's distance to the webcam, a reference hand scale L_{ref} is computed dynamically based on palm bone structure:

$$L_{\text{ref}} = \|P_0 - P_9\|_2$$

  where P_0 represents the wrist landmark and P_9 represents the middle finger metacarpophalangeal (MCP) joint.

- *Normalized Pinch Metric:* The distance from each fingertip to the thumb is scaled by L_{ref}:

$$d_i = \frac{\|P_i - P_4\|_2}{L_{\text{ref}}}, \quad \forall i \in \{8, 12, 16, 20\}$$

- *Involuntary Co-flexion Mitigation via* $\arg\min$*:* Because the middle, ring, and pinky fingers share deep flexor tendons, folding one finger inadvertently bends its neighbors. The system resolves this anatomical coupling using a constrained minimum selector:

$$i^* = \arg \min_{i \in \{8,12,16,20\}} (d_i)$$

  A pinch is categorized as a valid gesture for finger i^* if and only if:

$$d_{i^*} < \tau_{\text{pinch}} \quad \text{and} \quad d_{i^*} + \Delta < \min_{j \neq i^*}(d_j)$$

  where $\tau_{\text{pinch}} \approx 0.18$ represents the engagement threshold and $\Delta \approx 0.08$ denotes the safety boundary excluding whole-fist clenches.

2. **Debounce Filtering, Hysteresis, and Edge Triggering:** Webcam video streams at 30 FPS can generate dozens of duplicate triggers during a single physical tap. To eliminate jitter and multiple inputs:

   - *Rising-Edge Detection:* Implements a two-state machine (`RELEASED` $\rightarrow$ `PINCHED`). A character token is recorded solely during the instantaneous state transition from `RELEASED` to `PINCHED`.
   - *Hysteresis Release Threshold:* Once triggered, the state cannot return to `RELEASED` until the finger separates beyond an expanded release boundary:

$$d_{i^*} > \tau_{\text{release}}, \quad \text{where } \tau_{\text{release}} = \tau_{\text{pinch}} \times 1.4$$

   - *Cooldown Guard:* Enforces a mandatory cooldown duration (150ms) between consecutive pinch events to maintain crisp signal boundaries.

3. **Hybrid Morse Decoder & Auto-Commit Delimiter Engine:** The decoding core combines intuitive physical gestures with temporal state machines:

   - *Primitive Signal Mapping:* Index–Thumb pinch emits a **Dot** (`.`); Middle–Thumb pinch emits a **Dash** (`-`). Inputs are appended sequentially to a local token queue (`Current_Buffer`).
   - *Inactivity Auto-Commit:* When no pinch event occurs for an adjustable timeout duration ($\tau_{\text{commit}} \approx 0.8$s), the system automatically freezes `Current_Buffer`, resolves the symbol via an internal International Morse Code lookup dictionary (A–Z, 0–9), dispatches the translated character, and clears the buffer.
   - *Dedicated Control Gestures:*

  - **Ring–Thumb Pinch (Space):** Dispatches a literal spacebar character to delimit words and flushes `Current_Buffer`.
  - **Pinky–Thumb Pinch (Backspace):** If `Current_Buffer` contains uncommitted dots or dashes, it pops the most recent symbol; if the buffer is empty, it issues a system-level Backspace keystroke.
- *Error Fallback:* If an invalid sequence not present in the dictionary is committed, the system discards the buffer and emits a distinct visual and acoustic warning to prevent corrupted text injection.

4. **Interactive Virtual Keyboard HUD & Real-Time Desktop GUI:**
   - *Annotated Video Overlay:* Renders live video with skeletal hand connections; connecting vectors between active fingertips dynamically switch from white to green upon achieving pinch threshold criteria.
   - *Dynamic QWERTY Virtual Keyboard:* Displays an interactive on-screen keyboard that visually illuminates the decoded character in real time.
   - *Morse Cheat Sheet HUD:* A docking side-panel containing the full International Morse chart for rapid user lookup during live operation.
   - *Runtime Calibration Panel:* Interactive sliders allow users to customize pinch sensitivity (τ_{pinch}) and auto-commit timeout ($\tau_{\text{commit}} \in [0.5\text{s}, 1.5\text{s}]$).
5. **OS-Level Keystroke Injection & Multi-Sensory Audio Synthesis:**
   - *System-Level Dispatcher:* Synthesizes hardware key events using `pynput`, directly piping decoded characters into whatever external application currently holds focus (e.g., Microsoft Word, Notepad, terminal shells, or web browsers).
   - *Harmonic Audio Feedback:* Generates synchronized sine waves at 800 Hz using `pygame`—a short pulse ($\approx$ 60ms) for Dots, an extended tone ($\approx$ 180ms) for Dashes, and a mechanical click for commits and deletions, enabling touchless, sight-free typing.

4. Proposed Tech Stack

Component	Library / Framework	Purpose
Language	Python 3.10+	Core application architecture and pipeline orchestration
Vision Engine	`opencv-python`, `mediapipe`	Video frame capture and real-time 3D hand landmark extraction
Logic & Math	`numpy`, `math`	Normalized Euclidean distance computation, argmin classification, and FSM management
GUI & HUD	`PyQt6` or `customtkinter`	Modern desktop UI dashboard, virtual keyboard display, and Morse reference HUD
System Triggers	`pynput` / `pyautogui`, `pygame`	OS-level keystroke injection and real-time Morse audio feedback synthesis

5. Team Work Division (4 Members)

- **Member 1: Vision Engine Engineer**
  - Configure OpenCV webcam streaming and integrate MediaPipe Hands.
  - Extract and manage 3D coordinate arrays for landmarks ($P_0, P_4, P_8, P_{12}, P_{16}, P_{20}$).
  - Optimize real-time capture on an independent worker thread to ensure steady 30+ FPS performance.
- **Member 2: Gesture Logic & Tracking Core**
  - Implement hand-normalized Euclidean distance calculations and the argmin pinch selector.
  - Construct the debounce mechanism, hysteresis thresholds, and rising-edge state detection logic.
  - Build the Morse decoding dictionary and the timeout-driven auto-commit engine.
- **Member 3: Desktop UI & Virtual Keyboard Specialist**
  - Design the main desktop application interface using PyQt6 or CustomTkinter.
  - Implement the virtual keyboard widget featuring dynamic key-press illumination.
  - Build the Morse code cheat sheet HUD and sensitivity configuration sliders.
- **Member 4: System Integration, Audio & Documentation**
  - Integrate OS-level keystroke injection using `pynput`/`pyautogui`.
  - Set up real-time Morse audio feedback beeps using `pygame`.
  - Manage inter-thread queue communication, compile technical documentation, and coordinate the presentation demo.

6. Project Roadmap

- **Week 1:** Setup repository, establish webcam pipeline, extract 21 hand landmarks via MediaPipe, and verify 30+ FPS baseline stability.
- **Week 2:** Implement normalized pinch distance math, argmin classification, debounce filtering, and draft initial UI mockups.
- **Week 3:** Connect Morse translation FSM to UI, integrate OS-level keystroke dispatcher, and embed audio feedback triggers.
- **Week 4:** Edge-case fine-tuning (co-flexion threshold adjustments, occlusion handling), end-to-end integration testing, report finalization, and live demo practice.